

ESTUDIO — Carpeta validation/

Material de estudio para el final del TFC Bajoneá. Basado en el código real de `backend/src/main/java/com/bajonea/backend/validation/` — 12 anotaciones custom + 12 validadores.

Qué hay acá y por qué existe

Bean Validation (el estándar de Java para validar datos) trae anotaciones listas: `@NotBlank` , `@Email` , `@Size` , `@Pattern` , etc. Con eso alcanza para la mayoría de los casos.

Pero hay reglas que una anotación estándar no puede expresar. Ejemplo: validar un CUIT no es solo "11 dígitos" — hay que calcular su **dígito verificador** con el algoritmo módulo 11 de AFIP. Eso requiere código, no un regex.

Para esos casos se crean **anotaciones custom**, y esta carpeta las contiene.

La regla del proyecto: solo en los DTOs, nunca en las entities

Está escrito en el `package-info.java` de la propia carpeta:

"Uso exclusivo en los DTOs de request, nunca en las Entities de JPA. Las Entities representan el dato ya persistido y válido; la validación de entrada ocurre en el borde HTTP, sobre el DTO."

Por qué: la validación es **una decisión de borde**. Lo que entra desde afuera es sospechoso y hay que revisarlo; lo que ya está en la base ya pasó por ese filtro. Poner validaciones en las entities las ejecutaría en cada `save()` , incluso en operaciones internas que no vienen de un usuario.

Cómo funciona una anotación custom — el mecanismo

Siempre son dos archivos:

1. La anotación (validation/annotations/)

```
@Target(ElementType.FIELD)           // se pone sobre un campo
@Retention(RetentionPolicy.RUNTIME)  // vive en tiempo de ejecución
@Constraint(validatedBy = CuitValidator.class) // quién la valida
@Documented
public @interface ValidarCuit {
    String message() default "CUIT inválido"; // qué se muestra si falla
    Class<?>[] groups() default {};           // obligatorios del estándar
    Class<? extends Payload>[] payload() default {};
}
```

Elemento	Qué significa
@Target(FIELD)	Se puede poner sobre un campo. <code>TYPE</code> sería sobre la clase entera.
@Retention(RUNTIME)	La anotación sobrevive a la compilación y se puede leer mientras el programa corre. Sin esto, no funcionaría.
@Constraint(validatedBy = X)	Le dice a Bean Validation qué clase hace el trabajo real.
message()	El texto de error por defecto. Se puede pisar al usarla: <code>@ValidarCuit(message = "otro texto")</code> .
groups() y payload()	Los exige el estándar. En este proyecto no se usan, van vacíos.

2. El validador (validation/validators/)

```
public class CuitValidator implements ConstraintValidator<ValidarCuit, String> {
    @Override
    public boolean isValid(String cuIt, ConstraintValidatorContext context) {
        // devuelve true si es válido, false si no
    }
}
```

Los dos genéricos son: **qué anotación implementa** y **qué tipo de dato valida**.

El patrón "tolerante a null" — clave para entender el diseño

Fijate que **todos** los validadores del proyecto empiezan igual:

```
if (valor == null) { return true; } // o: if (valor == null || valor.isBlank())
```

Un validador que recibe `null` **devuelve true** (válido). Parece raro, pero es correcto:

La obligatoriedad la maneja @NotBlank / @NotNull , no el validador de formato. Cada anotación tiene una sola responsabilidad. Por eso los campos llevan las dos:

```
@NotBlank(message = "El CUIT es obligatorio") // se encarga de que exista
@ValidarCuit                                  // se encarga de que sea válido
private String cuIt;
```

LAS 12 ANOTACIONES CUSTOM

1. @ValidarCuit — el más complejo

Qué valida: que sea un CUIT de 11 dígitos con dígito verificador correcto según el algoritmo módulo 11 de AFIP.

Dónde se usa: RegistroComercioRequestDTO.cuit (1 uso).

El algoritmo, explicado paso a paso

```
private static final int[] MULTIPLICADORES = {5, 4, 3, 2, 7, 6, 5, 4, 3, 2};
```

- Chequea que sean exactamente **11 dígitos** (`\d{11}`).
- Multiplica cada uno de los **primeros 10 dígitos** por su multiplicador correspondiente.
- Suma** todos los productos.
- Calcula `resto = suma % 11` .
- `digitoEsperado = 11 - resto` .
- Dos casos borde:**
 - Si da **11** → el dígito esperado es **0**.
 - Si da **10** → **no existe un CUIT válido** con esa combinación → devuelve `false` .
- Compara el dígito esperado contra el **dígito 11** del CUIT ingresado.

Por qué esto importa (buena respuesta de mesa)

Un dígito verificador es un **mecanismo de detección de errores de tipeo**. Está calculado a partir de los otros dígitos, así que si te equivocás en uno solo, el verificador ya no coincide.

Lo que consigue: que `20-12345678-9` con un número mal escrito sea rechazado **antes** de guardarse, en vez de descubrirlo meses después cuando el comercio no puede facturar. Un `@Pattern` que solo verifique "11 dígitos" aceptaría cualquier número inventado.

2. @ValidarFormatoDni

Qué valida: que el DNI tenga **7 u 8 dígitos**, sin separadores.

```
if (dni == null || dni.isBlank()) { return true; }
return dni.matches("\\d{7,8}");
```

Dónde se usa: RegistroClienteRequestDTO.dni y RegistroComercioRequestDTO.dniRepresentante (2 usos).

Por qué 7 u 8: los DNI viejos tienen 7 dígitos, los actuales 8.

Importante: el DNI ya llega sanitizado — el setter manual del DTO le sacó puntos, guiones y espacios antes de que este validador lo vea (`"12.345.678"` → `"12345678"`).

Un detalle honesto: había otra anotación que se eliminó

Antes existía también `@ValidarDni` , más estricta: además del formato, exigía que el número estuviera en el rango `[1.000.000, 99.999.999]` . Se migró todo a `@ValidarFormatoDni` (que es tolerante a blanco y no chequea rango), y `@ValidarDni` quedó **sin ningún uso real**. Se verificó con un `grep` completo del proyecto —incluidos los tests— y **se eliminó junto con su validador**.

(Es un buen ejemplo de higiene de código para mencionar: se confirmó que no se usaba antes de borrarla, no se asumió.)

3. @ValidarFormatoEmail

Qué valida: formato de email — `algo@algo.algo` , sin espacios.

```
Pattern.compile("^^[\\s@]+@[\\s@]+\\.^[\\s@]+$");
```

El regex en criollo: "uno o más caracteres que no sean espacio ni `@` ", después una `@` , después lo mismo, después un punto, después lo mismo.

Dónde se usa: RegistroClienteRequestDTO.email , RegistroComercioRequestDTO.email , ComercioPerfilRequestDTO.emailContacto (3 usos).

Por qué se hizo una anotación propia si existe @Email

Dos razones, y las dos son buenas respuestas:

- `@Email` de Jakarta acepta cosas raras. Su implementación sigue el RFC al pie de la letra, y el RFC permite direcciones que en la práctica no existen (como `usuario@localhost` , sin punto). Este regex es más pragmático.

2. **Es tolerante a blanco.** `@Email` combinado con `@NotBlank` sobre un valor vacío hacía que los dos mensajes compitieran. Este validador devuelve `true` con blanco, así que gana el mensaje correcto ("es obligatorio").

Reemplazó a una combinación previa de `@Email` + `@Pattern` , que era redundante y no era blanco-tolerante en conjunto.

4. @ValidarFormatoNombre

Qué valida: que un nombre o apellido tenga solo letras del alfabeto español, espacios, guiones y apóstrofes.

```
Pattern.compile("^[A-Za-zÁÉÍÓÚáéíóúÑñÜü]+(?:['- ][A-Za-zÁÉÍÓÚáéíóúÑñÜü]+)*$");
```

El regex en criollo: "una o más letras, y después opcionalmente (un guion, apóstrofo o espacio, seguido de una o más letras) repetido las veces que haga falta".

Lo que acepta: "Juan" , "María José" , "García-López" , "O'Brien" , "Ñoño" .

Lo que rechaza: "Juan123" (números), "Juan@" (símbolos), "Juan Carlos" (doble espacio) y "-Juan" (empieza con separador).

Dónde se usa: los 4 campos de nombre/apellido de los dos registros (cliente y representante del comercio).

Por qué no permite números: un nombre propio no los lleva, y aceptarlos habilitaría datos basura.

5. @ValidarNombrePropio — el hermano estricto del anterior

Qué valida: casi lo mismo, pero usando letras Unicode y rechazando el vacío.

```
Pattern.compile("^[\\p{L}[\\p{L} ']*$");

if (valor == null) { return true; }
if (valor.trim().isEmpty()) { return false; } // <-- la diferencia
```

Dónde se usa: `ClienteEditarPerfilRequestDTO.nombre` y `.apellido` (2 usos).

La diferencia con @ValidarFormatoNombre — vale la pena tenerla clara

	@ValidarFormatoNombre	@ValidarNombrePropio
Alfabeto	Lista explícita de letras españolas	<code>\\p{L}</code> — cualquier letra Unicode
Con blanco	Devuelve <code>true</code> (delega en <code>@NotBlank</code>)	Devuelve <code>false</code>
Dónde se usa	Los registros	La edición de perfil de cliente

`\\p{L}` es una **clase Unicode**: matchea cualquier letra de cualquier idioma. Es más flexible que listar las letras a mano.

Que existan las dos es un residuo real de la evolución del proyecto: `@ValidarNombrePropio` es la original; `@ValidarFormatoNombre` es la versión nueva, blanco-tolerante, que se creó cuando se resolvió el problema de los mensajes que competían. La migración se hizo en los registros pero **no en la edición de perfil**.

(Si te preguntan, decilo así: es una inconsistencia menor conocida, con explicación histórica, no un descuido. Es más sólido que inventar una justificación.)

6. @ValidarTelefonoArgentino — el que documenta una decisión de diseño

Qué valida: un teléfono argentino con el **prefijo fijo** `+549` + exactamente 10 dígitos.

```
Pattern.compile("^[\\+549\\d{10}$");

String sanitizado = telefono.replaceAll("[ ()\\-]", "");
return PREFIJO_MAS_DIEZ_DIGITOS.matcher(sanitizado).matches();
```

Cómo funciona:

1. Descarta espacios, paréntesis y guiones (los tolera como separadores de tipeo).
2. Exige que empiece con `+549` .
3. Exige exactamente 10 dígitos después.

Dónde se usa: 5 campos de teléfono en todo el proyecto.

La decisión de diseño que hay detrás — muy buena para contar

Fijate lo que **NO** hace: si el valor viene sin el `+549` , **lo rechaza** en vez de agregárselo.

El comentario del código lo explica: *"un valor sin el prefijo se rechaza, no se completa silenciosamente (decisión explícita: el backend nunca corrige datos fuera de contrato)".*

Por qué: el propio frontend concatena el prefijo antes de enviar (la función `construirTelefono()` en `auth.js`), y el campo del prefijo en el HTML **no es editable**. Entonces un valor sin prefijo **solo puede venir de un cliente de API que no respeta el contrato**. En ese caso, corregirlo en silencio sería peor: estarías adivinando qué quiso decir alguien que ya está mandando datos mal formados.

La **versión anterior** era más **permisiva** (toleraba `054`, `15`, el `0` de larga distancia). Se endureció a propósito, y ese cambio **rompió tests de Playwright** que mandaban formatos viejos — fue una de las 7 regresiones que se detectaron y corrigieron al re-verificar la suite E2E.

7. @ValidarPasswordSegura

Qué valida: la política de contraseña del proyecto.

```
Pattern.compile("(?=[A-Z])(?=[a-z])(?=[\\d]).{8,72}$");
```

Qué exige:

Requisito	Cómo se expresa en el regex
Al menos una mayúscula	<code>(?=[A-Z])</code>
Al menos una minúscula	<code>(?=[a-z])</code>
Al menos un número	<code>(?=[\\d])</code>
Entre 8 y 72 caracteres	<code>.{8,72}</code>

Dónde se usa: 4 campos de contraseña (registro cliente, registro comercio, confirmar recuperación, cambio desde perfil).

Dos detalles técnicos que suman

- El `(?=...)` es un "lookahead". Significa *"mirá adelante y verificá que exista esto, pero no consumas caracteres"*. Es lo que permite exigir **tres condiciones simultáneas** en un solo regex, sin importar el orden en que aparezcan.
- El **límite de 72 no es arbitrario: es el límite real de BCrypt**. El algoritmo BCrypt **ignora todo lo que pase de 72 bytes**. Si aceptaras una contraseña de 100 caracteres, los últimos 28 no contarían para nada, y el usuario creería tener más seguridad de la que tiene. Poner el tope explícito es más honesto.
- No exige símbolo especial**, y fue una decisión consciente: se consideró excesivo para esta etapa. Es defendible — las guías modernas (NIST) priorizan la **longitud** sobre la complejidad de caracteres, porque las reglas muy estrictas empujan a la gente a escribir la contraseña en un papel.

Nota histórica: originalmente no exigía minúscula, aunque el documento de requisitos funcionales sí la pedía. Fue un gap real detectado y corregido.

8. @ValidarCodigoPostalArgentino

Qué valida: los dos formatos vigentes de código postal argentino.

```
FORMATO_CLASICO = Pattern.compile("^\\d{4}$"); // "9420"
FORMATO_CPA     = Pattern.compile("[A-Za-z]\\d{4}[A-Za-z]{3}$"); // "V9420ABC"
```

Dónde se usa: `DireccionRequestDTO.codigoPostal` (1 uso).

Formato	Ejemplo	Qué es
Clásico	9420	4 dígitos. El de toda la vida.
CPA	V9420ABC	Letra + 4 dígitos + 3 letras. El nuevo del Correo Argentino.

Acepta cualquiera de los dos, porque en la práctica la gente usa los dos indistintamente. Rechazar el clásico sería frustrante; rechazar el CPA sería no estar actualizado.

No distingue mayúsculas de minúsculas — la normalización a mayúsculas la hace `TextoUtils.normalizarCodigoPostal` después (ver ESTUDIO-UTIL.md).

9. @ValidarFechaNacimientoPlausible

Qué valida: que la fecha de nacimiento sea **razonable**: no futura y no anterior a 120 años.

```
private static final int ANTIGUEDAD_MAXIMA_ANIOS = 120;

if (fechaNacimiento.isAfter(hoy)) { return false; }
return !fechaNacimiento.isBefore(hoy.minusYears(ANTIGUEDAD_MAXIMA_ANIOS));
```

Dónde se usa: `RegistroClienteRequestDTO.fechaNacimiento` (1 uso).

Qué NO hace, y es lo importante: **no exige edad mínima**. Es un chequeo de *plausibilidad de datos*, no una política de edad. El propio comentario lo aclara: *"a diferencia de @MayorDeEdad, no exige ningún piso de edad"*.

10. @MayorDeEdad

Qué valida: que la persona tenga 18 años o más.

```
private static final int EDAD_MINIMA = 18;

if (fechaNacimiento.isAfter(LocalDate.now())) { return false; }
return Period.between(fechaNacimiento, LocalDate.now()).getYears() >= EDAD_MINIMA;
```

Dónde se usa: RegistroComercioRequestDTO.fechaNacimientoRepresentante — un solo campo.

La asimetría con el cliente — pregunta muy probable

Campo	Anotación	Exige 18 años
RegistroClienteRequestDTO.fechaNacimiento	@ValidarFechaNacimientoPlausible	No
RegistroComercioRequestDTO.fechaNacimientoRepresentante	@MayorDeEdad	Sí

Es intencional, no un descuido de sincronización. El razonamiento:

- El representante legal de una empresa debe ser mayor de edad — es un requisito legal real para firmar en nombre de una sociedad.
- Un cliente que pide comida no tiene por qué serlo. Un adolescente puede pedir una pizza.

Está documentado explícitamente en el javadoc del DTO como decisión de negocio. Si te preguntan por qué son distintas, esa es la respuesta.

Detalle técnico: Period.between(...).getYears() calcula la edad correctamente teniendo en cuenta el día y el mes. Si naciste el 31 de diciembre de 2007, el 30 de diciembre de 2025 todavía tenés 17 — no hace la cuenta grosera de restar años.

11. @ValidarUrlCloudinary — el más usado y el más matizado

Qué valida: que la URL sea https y su host sea exactamente res.cloudinary.com .

```
private static final String HOST_ESPERADO = "res.cloudinary.com";

URI uri = new URI(url);
return "https".equalsIgnoreCase(uri.getScheme()) && HOST_ESPERADO.equals(uri.getHost());
```

Dónde se usa: 7 campos — es la más usada del proyecto (fotos de perfil de comercio y usuario, imágenes de producto, recortes, foto en el registro).

Por qué usa URI y no un regex — muy buen detalle

Usar new URI(url).getHost() en vez de un matches("https://res\\.cloudinary\\.com.*") evita ataques de suplantación de dominio. Mirá estos casos:

URL maliciosa	¿Un regex ingenuo la aceptaría?	¿ getHost() la acepta?
https://res.cloudinary.com.atacante.com/foto.jpg	Sí (empieza con el texto esperado)	No — el host es res.cloudinary.com.atacante.com
https://atacante.com/res.cloudinary.com/foto.jpg	Depende del regex	No — el host es atacante.com
https://evilres.cloudinary.com/foto.jpg	Podría	No — el host no coincide exactamente

Parsear la URL de verdad y comparar el host exacto es lo correcto. Comparar strings sobre una URL es un error de seguridad clásico.

Y exige https , no http : las imágenes viajan por conexión cifrada.

La limitación que hay que saber decir

Esta anotación valida el DOMINIO, no la PROPIEDAD. O sea: verifica que la URL sea de Cloudinary, pero no que esa imagen sea tuya. Un comercio podría pegar la URL de una foto subida por otro comercio y pasaría la validación.

Cómo se mitiga ese hueco: con el flujo de subida firmada. CloudinaryService genera firmas acotadas a una carpeta derivada del comercioId o el productoId sacados del JWT. Sin una firma válida no podés subir nada a esa carpeta.

Y por eso ComercioPerfilRequestDTO no incluye fotoPerfilUrl1 : la foto tiene su propio par de endpoints con firma, precisamente para no depender solo de esta anotación.

"La anotación cubre el dominio; la propiedad la garantiza el flujo de firma, no la validación" — esa es la respuesta si te cuestionan el hueco.

12. @DireccionExclusionMutua — la que está declarada pero no se usa

Qué valida: a nivel de clase (no de campo), que un DTO tenga exactamente uno de clienteId / comercioId — nunca los dos, nunca ninguno.

Cómo está implementada — la única que usa reflexión

```
CampoLeido clienteId = leerCampo(dto, "clienteId");
CampoLeido comercioId = leerCampo(dto, "comercioId");

if (!clienteId.encontrado() && !comercioId.encontrado()) { return true; } // fail-open

return clientePresente ^ comercioPresente; // XOR
```

Tres cosas destacables:

1. `@Target(ElementType.TYPE)` — se pone sobre la clase, no sobre un campo. Es la única así, porque tiene que ver **dos campos a la vez**.
2. **Usa reflexión** (`getDeclaredField`) para leer los campos por nombre, recorriendo toda la jerarquía de clases. Así funciona con cualquier DTO que tenga esos dos campos, sin acoplarse a una clase puntual.
3. El **operador `^` es XOR** (*o exclusivo*): devuelve `true` solo si **exactamente uno** de los dos es verdadero. Es la forma más limpia de expresar "uno u otro, pero no ambos".
4. Es **"fail-open"**: si el DTO no declara ninguno de los dos campos, devuelve `true` . El criterio es que en ese caso el problema sería un error de configuración del DTO, no un dato inválido del usuario.

Lo importante: hoy no se aplica en ningún DTO

Verificado con `grep` sobre todo `dto/` : `@DireccionExclusionMutua` aparece **una sola vez**, y es **dentro de un comentario javadoc** de `DireccionRequestDTO` , explicando por qué **no** se aplica ahí. No hay ningún DTO anotado con ella.

El motivo está en ese mismo comentario: `DireccionRequestDTO` **no tiene** los campos `clienteId` ni `comercioId` . A quién pertenece la dirección lo decide el flujo:

- Venís por el registro de cliente → el service la asocia al cliente.
- Venís por el registro de comercio → la asocia al comercio.

La exclusión mutua queda garantizada estructuralmente, por tener dos flujos de registro separados en vez de un endpoint genérico de alta de direcciones. **No hay forma de mandar las dos cosas, porque los campos directamente no existen en el DTO.**

Cómo contarlo si te preguntan: *"La anotación existe y funciona, pero hoy no se usa. La regla se garantiza por diseño: como no hay un endpoint genérico de alta de direcciones, el flujo mismo determina el dueño y no hay ningún campo que el usuario pueda manipular. La anotación quedaría disponible si mañana se agregara ese endpoint genérico."*

Es más sólido decir eso que afirmar que está en uso. Y de hecho quedó documentado en el cierre de la Fase 14 como uno de los puntos "sin forma real de ejercitarlo" en los tests.

Tabla resumen de las 12 anotaciones

Anotación	Target	Qué valida	Usos reales
@ValidarUrlCloudinary	FIELD	URL <code>https</code> con host exacto <code>res.cloudinary.com</code>	7
@ValidarTelefonoArgentino	FIELD	+549 + 10 dígitos exactos	5
@ValidarPasswordSegura	FIELD	8-72 chars, mayúscula + minúscula + número	4
@ValidarFormatoNombre	FIELD	Letras españolas, espacios, guiones, apóstrofes	4
@ValidarFormatoEmail	FIELD	<code>algo@algo.algo</code> , sin espacios	3
@ValidarNombrePropio	FIELD	Letras Unicode; rechaza el vacío	2
@ValidarFormatoDni	FIELD	7 u 8 dígitos, ya sanitizado	2
@ValidarCuit	FIELD	11 dígitos + dígito verificador AFIP	1
@ValidarCodigoPostalArgentino	FIELD	4 dígitos, o CPA de 8	1
@ValidarFechaNacimientoPlausible	FIELD	No futura, no más de 120 años	1
@MayorDeEdad	FIELD	18 años o más	1
@DireccionExclusionMutua	TYPE	Exactamente uno de <code>clienteId</code> / <code>comercioId</code>	0 (garantizada por diseño)

Dónde va cada tipo de validación — el mapa completo

Esta tabla resume el criterio de todo el proyecto, y es muy útil si te preguntan "¿cómo decidís dónde poner una validación?":

Tipo de regla	Dónde va	Ejemplo
Obligatoriedad	@NotBlank / @NotNull en el DTO	@NotBlank en email
Formato simple	Anotación estándar	@Size(max = 100)
Formato con algoritmo	Anotación custom	@ValidarCuit
Dos campos del mismo objeto, un solo lugar	El service	direccionId obligatorio si es domicilio
Dos campos, varios lugares	Clase utilitaria	ComercioValidaciones
Requiere consultar la base	El service	Que el email no exista ya
Unicidad, garantía final	UNIQUE de la base	El email en usuario

Preguntas típicas de mesa

"¿Cómo validás los datos que entran?" Con Bean Validation: anotaciones sobre los campos del DTO, disparadas por @Valid en el controller. Uso las estándar donde alcanzan, y 12 anotaciones custom para reglas que un regex no puede expresar.

"Mostrame una validación custom." @ValidarCuit . No solo chequea que sean 11 dígitos: calcula el dígito verificador con el algoritmo módulo 11 de AFIP y lo compara. Así un CUIT con un número mal tipeado se rechaza antes de guardarse.

"¿Por qué los validadores devuelven true cuando el valor es null?" Porque la obligatoriedad es responsabilidad de @NotBlank / @NotNull , no del validador de formato. Cada anotación tiene una sola responsabilidad. Si ambos rechazaran el vacío, los mensajes competirían y el usuario vería "formato inválido" cuando en realidad no escribió nada.

"¿Por qué validás la URL de Cloudinary parseando el URI en vez de con un regex?" Porque comparar strings sobre una URL es inseguro: https://res.cloudinary.com.atacante.com pasaría un regex ingenuo. Parseando el URI y comparando el host exacto, no.

"¿Esa validación garantiza que la imagen sea del comercio?" No, valida el dominio, no la propiedad. Eso lo garantiza el flujo de subida firmada, donde la firma está acotada a una carpeta derivada del id que sale del JWT.

"¿Por qué el representante del comercio tiene que ser mayor de edad y el cliente no?" Porque el representante legal de una sociedad debe serlo por requisito legal, y un cliente que pide comida no. Es una decisión de negocio explícita, documentada en el DTO.

Índice de archivos cubiertos en este documento

Anotaciones (validation/annotations/)

Archivo	Qué valida
ValidarCuit.java	CUIT con dígito verificador AFIP.
ValidarFormatoDni.java	DNI de 7 u 8 dígitos.
ValidarFormatoEmail.java	Formato de email, tolerante a blanco.
ValidarFormatoNombre.java	Nombre/apellido, tolerante a blanco.
ValidarNombrePropio.java	Nombre/apellido con Unicode, rechaza el vacío.
ValidarTelefonoArgentino.java	+549 + 10 dígitos.
ValidarPasswordSegura.java	Política de contraseña (8-72, mayúscula, minúscula, número).
ValidarCodigoPostalArgentino.java	Código postal clásico o CPA.
ValidarFechaNacimientoPlausible.java	Fecha de nacimiento razonable.
MayorDeEdad.java	18 años o más.
ValidarUrICloudinary.java	URL con host exacto de Cloudinary.
DireccionExclusionMutua.java	Cliente o comercio, nunca los dos (sin uso actual).

Validadores (validation/validators/)

Archivo	Implementa
CuitValidator.java	@ValidarCuit — algoritmo módulo 11.
FormatoDniValidator.java	@ValidarFormatoDni .
FormatoEmailValidator.java	@ValidarFormatoEmail .
FormatoNombreValidator.java	@ValidarFormatoNombre .
NombrePropioValidator.java	@ValidarNombrePropio .
TelefonoArgentinoValidator.java	@ValidarTelefonoArgentino .
PasswordSeguraValidator.java	@ValidarPasswordSegura — regex con lookaheads.
CodigoPostalArgentinoValidator.java	@ValidarCodigoPostalArgentino .
FechaNacimientoPlausibleValidator.java	@ValidarFechaNacimientoPlausible .
MayorDeEdadValidator.java	@MayorDeEdad — USA Period.between .
UrlCloudinaryValidator.java	@ValidarUrlCloudinary — parsea el URI.
DireccionExclusionMutuaValidator.java	@DireccionExclusionMutua — reflexión + XOR.

Otros

Archivo	Qué es
package-info.java	Documenta la regla: estas anotaciones van solo en los DTOs de request, nunca en las entities.